

01. 计算机“看到”的图像是怎样的？（从灰度图像和彩色图像两个角度来解释）

1. 对于灰度图像，计算机看到的图像是每一个分辨率单位上都是一个个数字（取值范围0-255），如 MNIST 数据集中灰度图片在机器视觉里面就是一个 18*18 的一层二维矩阵。
2. 对于彩色图像，是一个三通道（RGB），每一个通道有一个取值，所以是三层二维矩阵。

02. 为什么卷积神经网络相对于全连接神经网络更有助于计算机理解图像？

全连接网络在展开输入向量的时候会丢失二维特征，并且当输入的矩阵很大时候，会造成输入层与隐藏层全连接的维度灾难，这都是不利于计算机图像理解的。

03. 在卷积神经网络中，卷积运算和池化运算的意义分别是什么？

卷积运算意义是提取输入的特征，通过卷积核在输入上以指定的步长滑动，计算重叠区域的内积生成特征图，能够有效地提取输入数据中的重要特征。

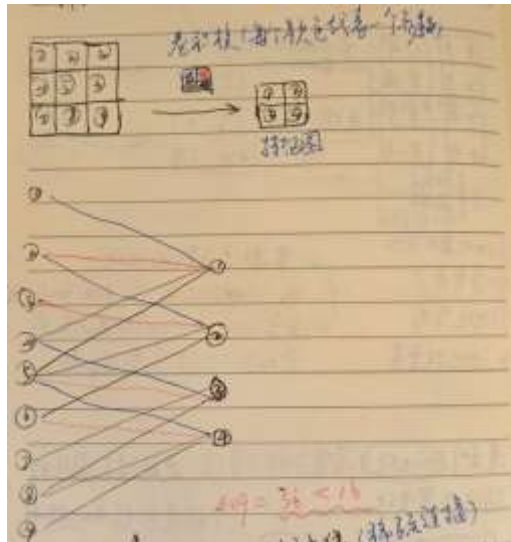
池化计算的意义是降低特征图的维度，一般是将特征图划分为若干个子区域，并对每个子区域进行统计汇总，常用的方法包括最大池化和平均池化。

04. 举例说明卷积层如何在输入发生平移时仍能正确地检测到相同的特征。



将“特征”平移后，卷积核也会遍历到特征的所有区域，就算平移到图像边缘，可以通过填充（padding）也能充分学习到边缘的特征，卷积运算的这种性质也即卷积运算的“平移不变性”

05. 举例解释什么是卷积神经网络中的稀疏连接。



稀疏体现在两方面：

- 1) 相比全连接神经网络，连接关系要稀疏 ($16 < 36$)
- 2) 参数稀疏：全连接参数需要 36 个参数，但是图中参数只有 4 个参数 (16 个连接公用“一套”参数，这一套参数即卷积核的参数)，这是参数的稀疏性

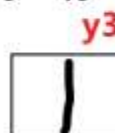
06. 根据视频 (023处理图像时的核心任务) 回答：只有与“边”相近的子图，对应元素乘积和才会较大吗？

答：不一定。下图中 y3 相比 y4 与边“1”更加相似，但是乘积上，y4 明显更大。

0	0	100	0	0
0	0	100	0	0
0	0	100	0	0
0	0	90	0	0
0	20	70	0	0



0	0	100	0	0	100	0	100	0	0
0	0	100	0	0	100	0	100	0	0
0	0	100	0	0	90	0	90	0	0
0	0	90	0	0	90	0	90	0	0
0	0	70	0	0	90	80	70	0	0



07. 根据视频 (025多通道单核卷积) 回答：多通道“单核”卷积中的卷积核的通道数与什么有关？

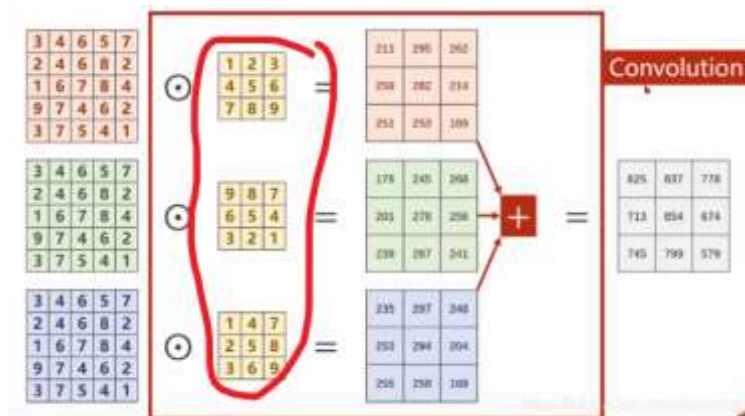
答：多通道单核卷积中的卷积核通道数与输入通道有关。比如处理彩色图片例子，彩色图片作为输入层，本身包含 R、G、B 三个通道，那么卷积核就需要三个独立的通道分别去处理 RGB 三个输入的通道。

08. 根据视频（025多通道单核卷积）回答：卷积核每个通道形状是否相同？

是的。因为需要对每个输入通道做相同尺寸的卷积，然后将结果在空间位置上逐点相加，形成一个输出通道。若形状不同，则无法对齐相加起来。

09. 根据视频（025多通道单核卷积）回答：卷积核每个通道填充数值是否相同？

不相同。在实际训练模型中，R、G、B 三个通道的卷积核参数即使初始相同，后面也会分化成不同的参数。再者，三个通道假如参数矩阵填充得都一样，那么就相当于对 RGB 三个通道用一种参数去学习（即把彩色图片当黑白图片处理），这不符合常理。



10. 根据视频（026多通道多核卷积）回答：

假如有一张大小为 $12p \times 12px$ 大小的png图片，用2个 3×3 大小的卷积核处理，

问题1: 得到了几个特征图？

问题2: 每个特征图的大小分别是多少？

问题3: “每个卷积核”对应多少个过滤器？

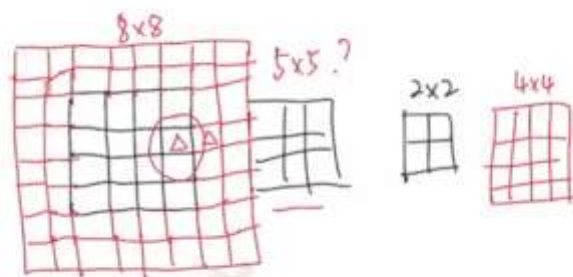
问题4: 所有卷积核一共有多少个参数？

答：因为题干说的是 3×3 大小的卷积核，但是未说明通道数，这里默认一个通道。因为三通道卷积核会说“ $3 \times 3 \times 3$ 大小的卷积核”

- 1) 得到 2 张特征图
- 2) 每个特征图大小为 10×10
- 3) 每个卷积核对应一个过滤器

4) 参数量=3*3*2=12 个

11. 根据视频（032卷积中的图像填充）中如图展示的内容回答：



0, 填充 p 步长 s 图像大小 i 输出特征图大小 o
思考: $0 \rightarrow i, p, s$

输出特征图大小o与填充p、步长s、图像大小i的关系是什么？

要得到公式必须得多加一个卷积核大小参数 k

$$O = \frac{I + 2P - K}{S} + 1$$

当 $S=1$, $P=0$ 的时候可以简化为

$$O = I - K + 1$$

12. 给定输入图像大小为 $(32 \times 32 \times 3)$ (高 x 宽 x 通道)，设计一个卷积神经网络，层设置如下：

- 卷积层1：卷积核大小 (3×3) ，步长 1，填充 1，输出通道数 16
- ReLU 激活层
- 池化层1：最大池化，池化窗口 (2×2) ，步长 2
- 卷积层2：卷积核大小 (5×5) ，步幅 1，填充 2，输出通道数 32
- ReLU 激活层
- 池化层2：最大池化，池化窗口 (2×2) ，步长 2

根据题目所给信息，计算每层的特征图尺寸和通道数、最后输出结果的尺寸和通道数以及该网络结构的参数量。

答：

	卷积层 1	池化层 1	卷积层 2	池化层 2
--	-------	-------	-------	-------

每层特征图尺寸	32*32	16*16	16*16	8*8
卷积核/池化核数量	16 个	16 个	32 个	32 个
每层通道数	3*16=48	16	32*3=96	32
每层参数量	16*3*3*3=432	2*2*16=64	32*3*3*3=864	2*2*32=128

最后输出结果的尺寸：8*8

最后输出结果的通道数：32

总网络参数量：432+64+864+128=1488 个参数

13. 以下是使用PyTorch进行模型训练的标准流程，根据代码内容回答问题（代码出自00703L同学理论作业第02周第09题）：

数据处理

模型构建

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

batch_size = 64
learning_rate = 0.01
momentum = 0.9
epochs = 10

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307, 0.0088)),
])

train_dataset = datasets.MNIST(root='.', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='.', train=False, download=True, transform=transform)

train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size, shuffle=False)

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 5, 1, 1)
        self.conv2 = nn.Conv2d(32, 64, 5, 1, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.dropout2 = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(1024, 100)
        self.fc2 = nn.Linear(100, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.dropout2(x)
        x = self.fc2(x)
        return F.log_softmax(x, dim=1)
```

得到“dataset”

由“dataset”得到“dataloader”

模型训练

```
model = net().to(device)

optimizer = optim.SGD(model.parameters(), lr=learning_rate, momentum=momentum,
weight_decay=5e-4)
criterion = nn.NLLLoss()

for epoch in range(epochs):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 100 == 0:
            print(f'Train Epoch: {epoch+1} [{batch_idx +
len(data)}/{len(train_loader.dataset)} ({100. * batch_idx / len(train_loader):.0f}%)]\tloss:
{loss.item():.6f}')
```

模型测试

```
model.eval()
test_loss = 0
correct = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model(data)
        test_loss += criterion(output, target).item()
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()
test_loss = len(test_loader.dataset)
print(f'\nTest set: Average loss: {test_loss:.4f}, Accuracy:
{correct}/{len(test_loader.dataset)} ({100. * correct / len(test_loader.dataset):.0f}%)\n')
```

③通常情况下，数据处理阶段最终目标是得到“dataloader”，为什么？有什么作用？

答：DataLoader 将数据分割成小批量（batches），使模型能够进行批量梯度下降，提高训练效率和内存利用率。通过 shuffle=True，在每个训练周期开始时打乱数据，可以设置多个 epoch 训练。

④使用PyTorch构建模型时，需将模型架构及前向传播过程定义为一个“类”，再由此“类”实例出一个模型对象，结合当前代码示例分析，此时模型对象中的属性都有哪些，这些属性与模型本身有什么关系？

答：

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.dropout2 = nn.Dropout2d(0.5)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)
```

卷积层，用于特征提取

Dropout层，用于防止过拟合

全连接层，用于分类

⑤结合代码示例，分析“optim”在模型训练过程中有哪些功能。

参数更新：使用SGD优化器，根据计算出的梯度更新模型的参数

lr=learning_rate设置学习率，控制参数更新的步长

momentum=momentum设置动量，帮助优化器加速收敛，避免在局部最小值处震荡。

```
optimizer = optim.SGD(model.parameters(), lr=learning_rate, momentum= momentum, weight_decay=5e-4)
```

权重衰减 (Weight Decay)
weight_decay=5e-4设置权重衰减，实现L2正则化，防止过拟合